

Number: P2327R1
Title: De-deprecating volatile compound operations
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: SG14, SG1, SG22, EWG, WG21
Date: 2021-09-15
Authors: Paul M. Bendixen <paulbendixen@gmail.com>
Contributors: Jens Maurer
Arthur O'Dwyer
Ben Saks
Email: paulbendixen@gmail.com
Reply to: Paul M. Bendixen

Revision history

Changes R0 → R1

Focus on bitwise compound expression solution, drop general dedeprecation of compound expressions.

1 Introduction

The C++ 20 standard deprecated many functionalities of the volatile keyword. This was due to P1152[Bastien, 2019]. The reasoning is given in the R0 version of the paper[Bastien, 2018].

The deprecation was not received too well in the embedded community as volatile is commonly used for communicating with peripheral devices in microcontrollers[van Ooijen, 2020].

The purpose of this paper is to give a solution that will not undo what was achieved with P1152, and still keep the parts that are critical to the embedded community.

2 Problem statement

2.1 Background

One of the great advantages of C++ is its closeness to the machine on which it operates. This enables C++ to be used in very constrained devices such as microcontrollers without any operating system. These systems often operate by doing bitwise manipulation on memory mapped registers.

While there are multiple ways to do bitwise manipulation in a memory location the most idiomatic way is something along the following lines:

```
// In vendor supplied hardware abstraction layer
struct ADC {
    volatile uint8_t CTRL;
    volatile uint8_t VALUE;
    ...
};
#define ADC_ENABLE (1 << ADC_CTRL_ENABLE)

// in user code
```

```
ADC1->CTRL |= ADC_ENABLE; // Start the ADC
...
ADC1->CTRL &= ~ADC_ENABLE; // Stop the ADC
```

Vendors supply hardware abstraction libraries (HALs) containing macros or inline functions for setting or clearing bits utilizing this idiom, or may use it in code created by code generators. The following example is from a library for the Energy Micro (now Silicon Labs) series of ARM microcontrollers:

```
// Copyright 2012 Energy Micro EMLib from em_i2c.h
__STATIC_INLINE void I2C_IntDisable(I2C_TypeDef *i2c, uint32_t flags)
{
    i2c->IEN &= ~(flags);
}
```

It is clear from the previous example that this idiomatic usage of compound operations clashes with the deprecation of compound operations on volatile. The vendors of these libraries are almost always the chip vendors themselves and they supply C libraries that are used by C++ developers. Based on experience they will not be likely to update their header files to conform to the C++ standard. The argument against compound operations on volatile variables is that it leads the programmer to believe that the operation itself becomes compounded and therefore atomic.

Now since there can be a read, modify, write cycle it is possible that an interrupt would happen in between the read and the write, however most embedded code that uses this idiom needs to take care of this by being right by construction i.e. not bit-twiddling variables that are used in the interrupt service routines (ISRs). The problem being that there is currently no way to do this in a guaranteed atomic manner (see also [Craig, 2020, section 3.4.1]).

The fact that the code must be correct by construction makes it seem that C++20 is deprecating fully functioning code.

2.2 Scope

So what is the impact of deprecating compound operations on volatile? A search of some of the more widely used embedded libraries show that, it is in some way used in all hardware libraries commonly used for embedded systems available today.

The numbers below are done by using UNIX `grep`¹ to find usages of either `|=` or `&=` on variables with a code style usually associated with volatile members in the code bases of the respective libraries. While this method isn't foolproof it does give an indication of the problem.

Library	compound operations
Silabs Gecko SDK	293
AVR libc 2.0	205
Raspberry pi Pico SDK	13

Table 1: Occurrences of compound operations on variables typically associated with volatile

This illustrates the major point, not only is there a lot of code already written out there working as expected but the usage of this idiom also prevents adopters of C++ to use the C libraries provided with their toolchains.

¹The search was done using the command
`grep -re "[A-Z_0-9]\+(\[.*\])\*[ ]\+[\&]= " -include *.h .`

Furthermore, one of the most common ways to gradually change from C to C++ is by keeping the same code and compile it with a C++ compiler. (See e.g. [von Tessin, 2019, from 27:46]). This procedure involves modifying the code to be correct in C++ and would possibly involve moving compound volatile statements to non-compound statements. However the prevalence of this idiom in vendor supplied C headers would prevent this low cost approach and would require a larger up front investment creating new headers to replace the vendor supplied ones.

The idea that businesses will be willing to spend the effort to update existing codebases that become defunct because of these changes seems unlikely, rather it seems likely that they will mandate using no newer standard versions.

2.3 Error potential

One of the arguments for deprecation in [Bastien, 2018] is the potential for error for programmers unaccustomed to using volatile. However removal of the compound operators will also risk the introduction of errors such as in the following.

```
UART1->UCSR0B |= (1<<UCSZ01); // (1)
UART1->UCSR0B = UART1->UCSR0B | (1<<UCSZ01); // (2)
UART2->UCSR0B = UART1->UCSR0B | (1<<UCSZ01); // (3)
```

The code in (1) is the original code, setting a bit in a mapped register. In (2) the code is transformed to the style that is suggested as a replacement. (3) describes a possible error scenario, where the device is changed to another, but due to the code duplication of the updated style, an error has slipped in and the value of the old device is read.

An error such as the above will not necessarily be caught in code review, and will possibly not even be found in immediate testing if UART1 happens to have the correct setting during testing, such code is also notoriously hard to unit test. As such the deprecation will trade errors where volatile is erroneously used to express atomicity for hard to discover and hard to detect errors due to duplication.

2.4 Didn't we just go over this?

While the proposal to deprecate was heard in committee meetings, the main focus was on problems arising with multi-threaded code (SG1) and with EWG, neither of these groups can be expected to be familiar with the inner workings on microcontrollers.

3 Solution

As the compound operations are mainly used for bitwise manipulation, the proposed solution is to only de-deprecate the use of bitwise compound operations (`|=` `&=` `^=`) as these are the ones that are useful for this purpose. In the examination of the libraries `+=` and `-=` did not occur, however they might in commercial / closed source libraries.

4 Wording

Change expr.ass paragraph 6

The behavior of an expression of the form `E1 op= E2` is equivalent to `E1 = E1 op E2` except that `E1` is evaluated only once. Such expressions are deprecated if `E1` has volatile-qualified type [and op is not one of the bitwise operators `|`, `&`, `^`](#); see [depr.volatile.type].

For `+=` and `-=`, E1 shall either have arithmetic type or be a pointer to a possibly cv-qualified completely-defined object type. In all other cases, E1 shall have arithmetic type.

Change `expr.ass` paragraph 5

~~A simple~~[An](#) assignment whose left operand is of a volatile-qualified type is deprecated ([`depr.volatile.type`]) unless the (possibly parenthesized) assignment is a discarded-value expression or an unevaluated operand.

Add the following to the examples in [`depr.volatile.type`] paragraph 2

```
brachiosaur |= neck; // OK Bitwise compound expression
```

5 Impact

The changes proposed by this paper would affect the current proposal P2139[Meredith, 2020].

6 Thanks

Thanks to Jens Maurer for the suggestion on the solution, for wording and presentation assistance.

Thanks to Wouter van Ooijen for starting this discussion.

Thanks to the entire SG14 group for feedback on initial drafts.

Bibliography

- [Bastien, 2018] Bastien, J. (2018). P1152r0 deprecating volatile. Technical Report P1152R0, ISO. Retrieved at [wg21.link/P1152R0](#).
- [Bastien, 2019] Bastien, J. (2019). P1152r0 deprecating volatile. Technical Report P1152R4, ISO. Retrieved at [wg21.link/P1152R4](#).
- [Craig, 2020] Craig, B. (2020). P2268r0 freestanding roadmap. Retrieved at [wg21.link/P2268R0](#).
- [Meredith, 2020] Meredith, A. (2020). P2139 reviewing deprecated facilities of c++20 for c++23. Technical Report P2139, ISO. Retrieved at [wg21.link/P2139](#).
- [van Ooijen, 2020] van Ooijen, W. (2020). Compound assignment to volatile must be undeprecated. https://www.reddit.com/r/cpp/comments/jswz3z/compound_assignment_to_volatile_must_be/.
- [von Tessin, 2019] von Tessin, M. (2019). C++ in deeply embedded systems. EmBo++ presentation <https://youtu.be/nuwOJ-xUhFU>.